Player Churn Prediction

Technical Report: Baseline Model and End-to-End Pipeline

Javier Herrero Pérez
Whizdom.ai

13 October 2026

Abstract

In this project I predict, for each active player of a brand, the chance that the player will stop betting for 60 days. With one command (`make pipeline AS_OF=date`) I read the S3 data lake (`org/40-gold`), I build 39 features, I train a calibrated LightGBM model and a Cox proportional hazards (Cox PH) model, I test them with bootstrap intervals, I run an 11 month backtest and I write a daily table of scores. In the backtest, the mean area under the ROC curve (AUC) is 0.87 to 0.91 in five brands, and this is between 0.11 and 0.13 higher than recency alone. If I run a campaign on the 1,000 players with the highest value at risk, I would cover 60% to 69% of the gross gaming revenue (GGR) at risk, instead of 6% to 13% with recency. Most of this gain comes from using money: a simple value × recency rule covers almost the same. The data goes from 2025-04-01 to 2026-10-06.

Contents

1 Summary **2**

2 Data **2**

3 Method **3**

4 Results **6**

5 Validation of the signals table **12**

6 Decisions against the plan **13**

7 Conclusions and limitations **13**

A How to reproduce the project **14**

B Abbreviations and terms **14**

List of Figures

1 Pipeline flow . . . 3
2 Definition of a cutoff . . . 3
3 Training, validation and test split . . . 5
4 ROC curves, brand 64 . . . 7
5 SHAP weight by feature family . . . 8
6 Calibration curves, brand 64 . . . 8
7 Backtest AUC with 95% intervals . . . 9
8 Model against recency . . . 9

9 Extra value of the model over the value × recency rule . . . 10
10 Churn rate and ECE by month . . . 11
11 Top-decile ratio and its maximum . . . 12
12 Failed checks in the signals table . . . 12

List of Tables

1 Data lake tables used in the project. . . . 3
2 False churn by threshold . . . 3
3 Feature families. . . . 4
4 Data-quality checks . . . 4
5 LightGBM training steps . . . 5
6 Registered model of each brand . . . 7
7 Backtest by brand . . . 8
8 GGR at risk reached in the backtest . . . 10
9 Problems in the signals table and their effect. . . . 13
10 Differences with the delivery plan. . . . 13
11 Main commands. . . . 14

1 Summary

- **Churn**: 60 days without a bet. The plan proposed 30 days, but then 17% to 24% of the churned players come back.
- **Model**: a calibrated LightGBM gives the 60-day probability. A Cox PH model gives the 7, 14 and 30-day horizons.
- **Money**: with the same 1,000 contacts per month, ranking by value at risk reaches 5 to 10 times more GGR at risk than recency. Most of this comes from using money: a simple value × recency rule reaches 94% to 97% of it, and the model adds 3% to 6% on top (Section 4.7).
- **Strongest predictor**: how often the player plays (`active_days_l90d`), then recency and tenure.
- **Backtest**: brands 23 and 73 pass all three conditions. Brands 14, 64 and 72 only fail the ECE in months with a seasonal change (December, and May in brand 14). The ranking of the players is not affected.
- **Source data**: recency and tenure are wrong in the signals table, so the pipeline computes them again from the daily activity.

The example figures are for brand 64 unless another brand is named. The result tables cover all brands.

2 Data

S3 access is in `src/features/datalake.py` (DuckDB on Parquet). `src/features/daily_cache.py` keeps local daily copies, so S3 is not read again on every run. The data-quality profile (T2) uses SQL aggregations on the six full tables, with no sampling (`eda/profile_tables.py`, reports in `docs/dq_reports/`).

Data limits:

- Completed deposits only exist from March 2026. Before that, the deposit features are empty, not 0.
- The history was computed again in a *backfill* that ended on 2026-08-08.
- There are only 18 months, with no full previous year to model seasonality.

Table 1: Data lake tables used in the project.

Table	Rows	Use
`gld_player_gaming_daily`	59.6 M	Days with a bet: the base of churn and recency
`gld_player_financial_daily`	688 M	Wagered, won, GGR and bonus per day
`gld_player_payments_daily`	5.6 M	Deposits and withdrawals; starts on 2025-09-18 and misses 40 days
`gld_player_signals_daily`	689 M	Precomputed signals; `brand_id` is empty before July 2026

3 Method

3.1 Pipeline

`src/features/run_pipeline.py` runs every step "as of" `AS_OF`: no later data is read. Each step uses the exact file or MLflow run made by the step before (Figure 1). Everything is logged in MLflow (`make mlflow-ui`).

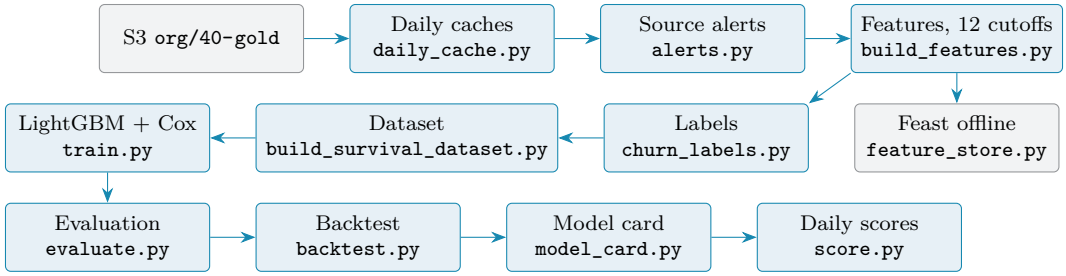


Figure 1: The flow of `make pipeline`. Each box shows the file that runs the step.

3.2 Churn and cutoffs (T3)

A player churns after 60 days without a bet. Each cutoff is the first day of a month. Its population is the players with a bet in the 30 days before, and its label looks at the next 60 days (Figure 2). Code: `src/features/churn_labels.py`; analysis: `eda/02_churn_definition.ipynb`.

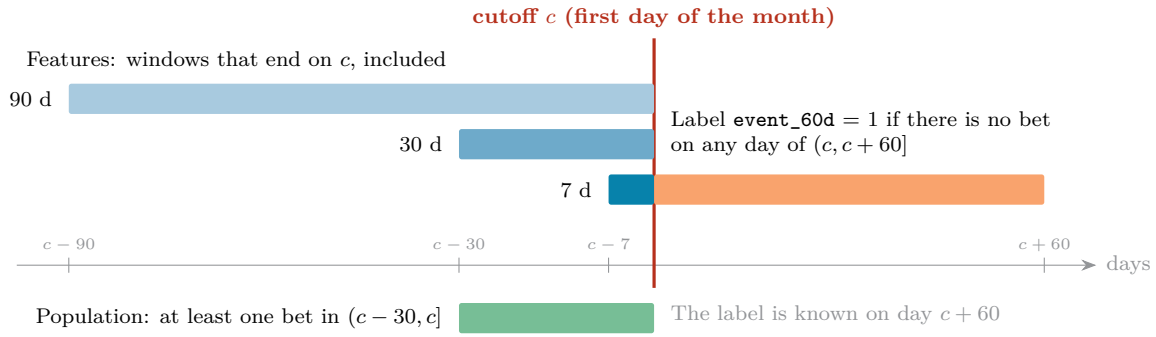


Figure 2: How a cutoff c is defined: the features look back, the label looks forward.

Table 2: False churn by threshold: players who bet again in the next 60 days.

Silent days	Brand 64	All brands	Reading
30	23.6%	17.0%	Too much false churn
60	**13.0%**	**9.6%**	First standard window below 15%
90	8.7%	6.5%	Only 4.3 points lower, for one more month of waiting

Why not 90 days. A label of N days is known N days after the cutoff. With 90 days the labels arrive one month later, the calibration is further behind the season (Section 4.8) and one cutoff with a complete label is lost. All this for only 4.3 points less false churn.

For player i in the population of cutoff c, the label is

$$y_{i,c} = \begin{cases} 1 & \text{if player } i \text{ places no bet on any day of } (c, c+60], \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

Labels. `event_60d` is $y_{i,c}$ in Equation (1) and is the target of LightGBM; `duration_days` and `event_observed` for Cox PH (censored if the silence does not fit in the data); and `churn_within_7d/14d/30d` to calibrate the Cox horizons. Survival labels follow the usual right-censoring setup of Kaplan and Meier (1958) and Cox (1972).

3.3 Features and feature store (T4 to T6)

Signal explorer (T4). I tested 32 hypotheses on 12 training cutoffs with four measures: lift, univariate Cox hazard ratio (Cox, 1972), single-signal AUC and stability across cutoffs. 21 were promoted and 11 rejected. Code: `eda/signal_explorer.py`.

Features (T5). 39 features in 8 families (Table 3), built with data up to the cutoff (`src/features/build_features.py`, dictionary in `docs/feature_dictionary.md`). Amounts are capped at the 99.5th percentile $q_{0.995}$ of each brand (`winsorisation.py`) and then scaled with a signed log:

$$\tilde{x} = \operatorname{sign}(x)\, \log\bigl(1 + |x|\bigr), \qquad x = \min\bigl(x_{\text{raw}},\, q_{0.995}\bigr), \tag{2}$$

where x_{raw} is the raw value of the feature. The caps are computed on the training cutoffs only. A test (`test_leakage.py`) checks that no feature uses data after the cutoff.

Table 3: Feature families.

Family	N	Examples
Recency	2	`days_since_last_bet`, `days_since_last_deposit`
Frequency	9	`active_days_l7d/l30d/l90d`, `bets_l30d`, `sessions_l7d`
Monetary	7	`wagered_eur_l7d`, `ggr_eur_l30d`, `deposits_eur_l30d`
Lag	3	`prior_wagered_eur_l7d`, `prior_active_days_l7d`
Trend	3	`wagered_trend_7`, `heavy_loss_multiple`
Tenure and life cycle	3	`days_since_first_bet`, `prior_dormancy_spells_14d`
Mix and behaviour	7	`engagement_score`, `games_breadth_30d`, `losing_streak`
Deposits	5	`deposited_within_3d/7d/14d`, `failed_deposits_14d`

Feature store (T6). The Feast definitions are in `src/features/feature_repo/definitions.py`. `src/features/feature_store.py` writes one Parquet file per brand to the offline store and checks the point-in-time join: Feast must return exactly what was written, and nothing before the first cutoff. It is local because I only have read access to AWS.

3.4 Data quality and alerts

Table 4: The four stages of the data-quality checks (limits in `configs/eda_alerts.yaml`).

Stage	What it checks	File
1 Quality	Pandera suites for each cache and for the features	`eda/expectations/`
2 Anomalies	IQR, MAD, Isolation Forest and daily totals; nothing is deleted, values are capped	`eda/anomalies.py`
3 Alerts	Rows, nulls, totals and PSI. A critical alert stops the pipeline; drift only sets `drift_flag`	`alerts.py`
4 Summary	Each statistic against the previous dataset of the brand	`eda/summary.py`

3.5 Models (T8 and T9)

All the training is in `src/models/train.py`. The models and their parameters are in `catalog/model_library_catalog.json`.

Table 5: LightGBM training steps. Each step is a nested MLflow run.

Step	What it does
Selection	Drops features with more than 40% nulls, almost constant or highly correlated (over 0.95); adversarial validation with a LightGBM; pruning by permutation importance (`select_features.py`)
Reference	LightGBM with default values: 63 leaves, learning rate 0.05, early stopping
Imbalance	`scale_pos_weight`, `is_unbalance` or nothing, chosen by log-loss after calibration
Optuna	100 trials (20 in the backtest) that minimise the validation log-loss
Regularisation	Only if the training AUC is more than 0.03 above the validation AUC
Calibration	Isotonic or Platt per brand, chosen by cross-validated ECE (`calibration.py`)

The **Cox PH** model (Cox, 1972), fitted with the Python library lifelines (Davidson-Pilon, 2019), uses the same features, is stratified by brand and calibrates its 7, 14 and 30-day horizons. The **evaluation** (`src/evaluation/evaluate.py`) gives AUC, ECE, log-loss and top-decile precision for LightGBM, and the concordance index (C-index) of Harrell et al. (1982), the integrated Brier score (IBS) and the time-dependent AUC for Cox. All metrics have 95% bootstrap intervals.

For the expected calibration error (ECE), the predictions are split into $B = 10$ bins of equal size, and

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{n} \left| \bar{y}_b - \bar{p}_b \right|, \tag{3}$$

where n_b is the number of players in bin b, $\bar{y}_b$ is their observed churn rate and $\bar{p}_b$ is their mean predicted probability.

3.6 Split and backtest (T10)

The registered model uses a time split: months 1 to 9 for training, month 10 for validation and months 11 and 12 for test. The rolling-origin backtest (`src/models/backtest.py`) trains again on each of the 11 first days of the month in a year, using only the cutoffs whose label was already known on that day (Figure 3). The month before the test is left out because its label is not closed yet: using it would mean using information that did not exist on that day.

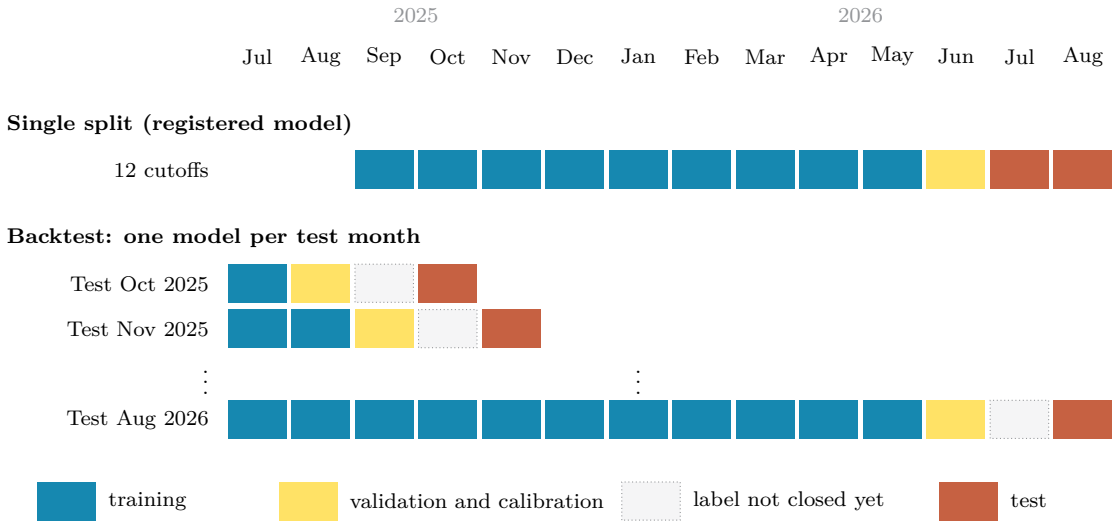


Figure 3: Training, validation and test (cutoffs of brand 73). In the backtest, the training data grows from 2 to 12 months.

Comparators: optimised LightGBM, reference LightGBM (to measure what tuning adds), Cox PH and recency alone.

The recency rule. Recency is the simple baseline that the model must beat. It is not a trained model: it uses one feature and no parameters. The risk score of player i at cutoff c is the number of

days since the last bet,

$$s_{i,c}^{\text{rec}} = c - t_{i,c}^{\text{last}}, \tag{4}$$

where $t_{i,c}^{\text{last}}$ is the last day with a bet up to the cutoff, taken from the daily activity (`days_since_last_bet`, not the broken column of the signals table). More days without a bet means more risk. Its AUC, its top-decile precision and its campaign replay are computed with this score in the same way as for the model. **Campaign replay**: at each cutoff, the 1,000 players with the highest value at risk are chosen, and I measure the share of the GGR of the real churners that they cover (the size is provisional until the CRM team confirms its capacity). The expected GGR of player i in the next 30 days is

$$G_i = \max\Big(0,\ \frac{30}{90}\,\text{GGR}_i^{(90)}\Big), \tag{5}$$

where $\text{GGR}_i^{(90)}$ is the GGR of the player in the last 90 days. In the backtest, the model ranks the players by the value at risk $V_i = G_i\, p_i^{(60)}$, where $p_i^{(60)}$ is the calibrated 60-day churn probability of LightGBM, and the GGR at risk reached by the campaign is the sum of $G_i\, y_{i,c}$ over the contacted players, with $y_{i,c}$ from Equation (1). The daily scores use the same G_i but multiply it by the Cox probability that the churn starts within 30 days (`value_at_risk_30d`).

Pass conditions: AUC ≥ 0.70 and ECE ≤ 0.08 in every month, and a top-decile ratio $R \geq 1.10$, where

$$R = \frac{\overline{\text{Prec}}_{10\%}(\text{model})}{\overline{\text{Prec}}_{10\%}(\text{recency})} \tag{6}$$

is the mean precision of the model in the riskiest 10% of players divided by the same value for the recency rule, Equation (4).

3.7 Daily scoring (T11 and T12)

`src/models/score.py` loads the registered model, builds the features with the same code as in training, runs the alerts, estimates the calibration level again and writes `data/03_output/player_scores/run_date=.../brand_id=N.parquet`. Running the same date again overwrites its partition. Main columns: `p_churn_60d`, `p_churn_7d/14d/30d`, `value_at_risk_30d`, `risk_band` (decile), `top_drivers` (TreeSHAP) and `drift_flag` (schema in `docs/schema_player_scores.md`). The **model card** (`src/models/model_card.py`) sums up each model in one HTML page, attached to its version in MLflow.

4 Results

4.1 Model of each brand

- The models use 5 to 7 features. The 90-day frequency appears in four of the five brands.
- The Cox PH model of brand 64 has a C-index of 0.871 [0.867, 0.875] and an IBS of 0.036.
- **Tuning adds nothing**: the reference and the optimised configurations give the same AUC (0.8786 and 0.8785), in earlier windows and in the backtest.
- Brand 14 is the hardest: 28% to 62% of its active players are new each month, and 68% to 90% of them churn.

4.2 ROC curves

Figure 4 shows the ROC curves of the registered model of brand 64. On the left, the training, validation and test curves are almost on top of each other (AUC 0.886, 0.886 and 0.895), so the model does not overfit. On the right, on the most recent cutoffs (July and August 2026), the model is far above recency alone (AUC 0.747).

Table 6: Registered model of each brand, test in July and August 2026.

Brand	Version	Selected features	AUC	ECE
14	v1	`active_days_l90d`, `prior_dormancy_spells_14d`, `sessions_l7d`, `wagered_eur_l90d`, `wagered_eur_l7d`, `segment_1`, `bets_l30d`	0.909	0.013
23	v1	`days_since_last_bet`, `active_days_l30d`, `prior_dormancy_spells_14d`, `prior_ggr_eur_l30d`, `losing_streak`, `days_since_last_return`	0.919	0.035
64	v63	`active_days_l90d`, `days_since_last_bet`, `days_since_first_bet`, `wagered_eur_l7d`, `engagement_score`	0.895	0.023
72	v1	`active_days_l90d`, `prior_dormancy_spells_14d`, `sessions_l7d`, `prior_wagered_eur_l7d`, `wagered_eur_l7d`	0.869	0.024
73	v1	`active_days_l90d`, `days_since_last_bet`, `prior_dormancy_spells_14d`, `sessions_l7d`, `active_days_l30d`, `days_since_last_return`	0.902	0.007

Note. AUC: area under the ROC curve. ECE: expected calibration error, Equation (3). Source: MLflow runs of each registered version.

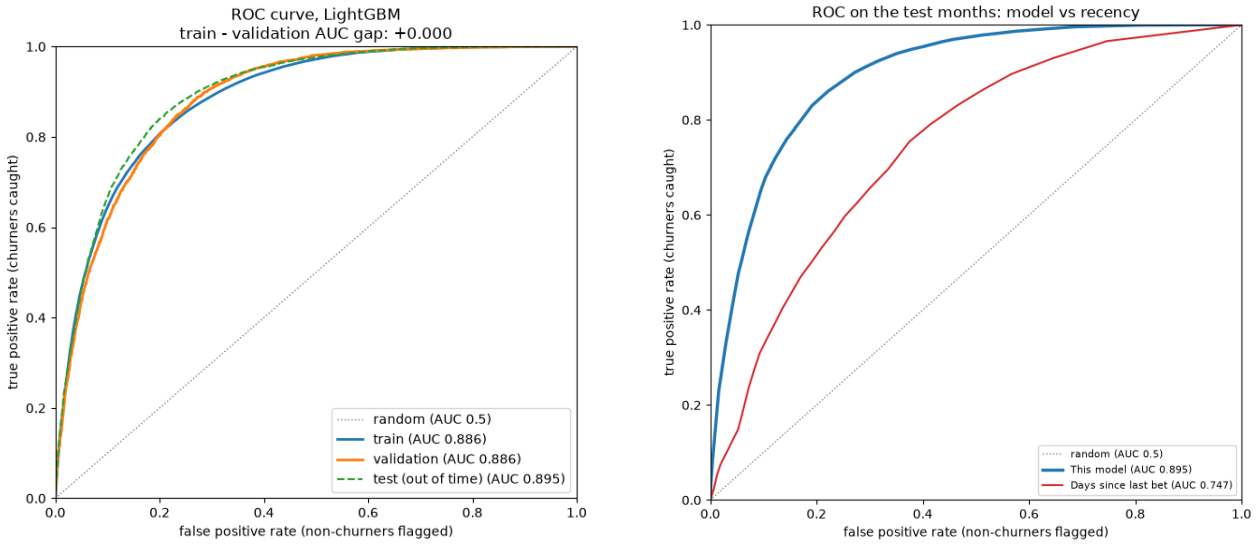


Figure 4: ROC curves of the registered model of brand 64 (`v63`). Left: training, validation and test. Right: the model against recency on the test months (July and August 2026).

4.3 Which features matter most

To see all eight families, I trained an analysis LightGBM with the 39 features (test AUC 0.898, the same as the registered model) and added up its SHAP values by family (Figure 5). Frequency brings 51% of the weight. Tenure, recency and monetary features bring 12% to 13% each, and the rest between 1% and 6%. This is why the selection keeps only a few features without losing AUC.

4.4 Calibration

Without calibration, LightGBM predicts too much churn, because `scale_pos_weight` gives more weight to the players who leave. Training tests two calibrations, isotonic and Platt, and keeps the one with the lowest cross-validated ECE on the validation month (Figure 6). In validation, isotonic wins (ECE 0.004 against 0.011). In test the two are close (0.023 and 0.017): churn went down from 38.1% in June to 36.4% in July and August, and no calibration learnt in June can know that. The test is only reported; it is never used to choose.

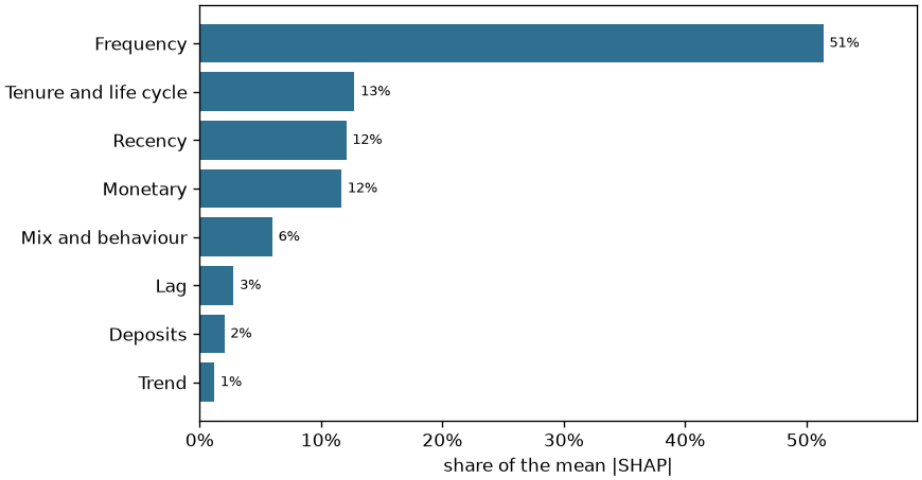


Figure 5: Share of the mean |SHAP| of each family (brand 64, validation month).

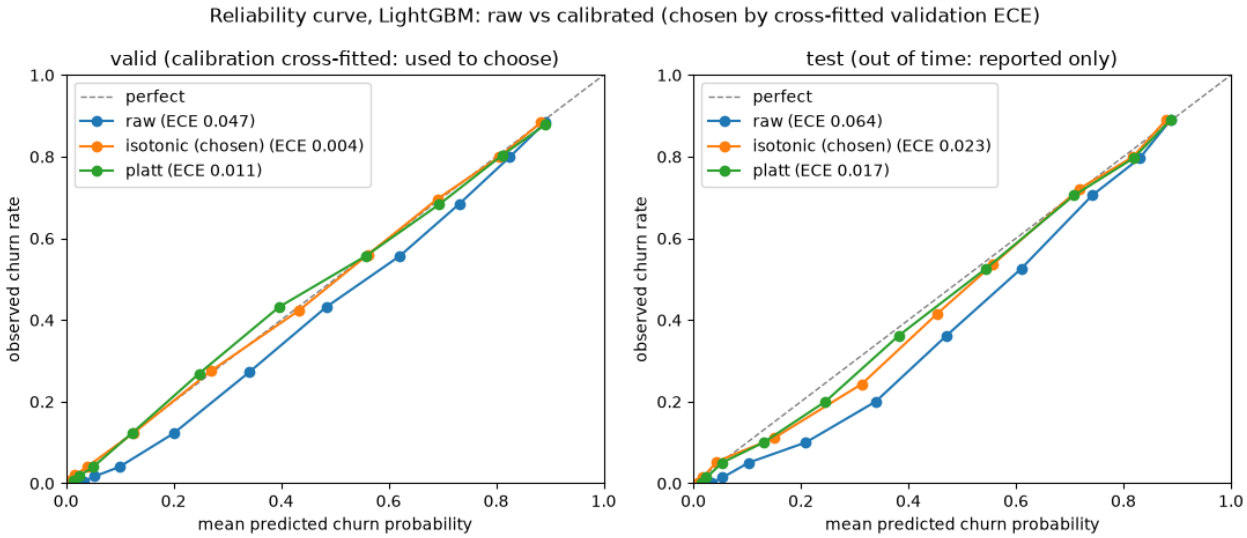


Figure 6: Calibration curves of the registered model of brand 64 (`v63`), as saved in MLflow: raw, isotonic (chosen) and Platt. Left: validation; right: test.

4.5 Backtest

Table 7: Backtest by brand: means over the 11 monthly test cutoffs, with 95% intervals.

Brand	Model AUC	Recency AUC	Cox C-index	Ratio R	Max ECE	GGR covered	Verdict
14	0.870 [0.853, 0.889]	0.743 [0.718, 0.772]	0.839 [0.817, 0.859]	1.19 [1.13, 1.25]	0.112	69% (13%) [63%, 74%]	FAIL
23	0.912 [0.903, 0.921]	0.797 [0.782, 0.809]	0.841 [0.829, 0.859]	1.27 [1.21, 1.36]	0.066	62% (9%) [58%, 65%]	PASS
64	0.875 [0.867, 0.882]	0.752 [0.733, 0.767]	0.815 [0.802, 0.834]	1.21 [1.17, 1.25]	0.087	66% (11%) [62%, 70%]	FAIL
72	0.870 [0.861, 0.880]	0.738 [0.718, 0.754]	0.833 [0.823, 0.845]	1.16 [1.12, 1.20]	0.099	64% (10%) [60%, 69%]	FAIL
73	0.887 [0.880, 0.895]	0.774 [0.763, 0.782]	0.828 [0.817, 0.845]	1.24 [1.20, 1.29]	0.042	60% (6%) [57%, 63%]	PASS

Note. Values in brackets are 95% bootstrap intervals of the mean, with the 11 months resampled 200 times. AUC: area under the ROC curve. C-index: concordance index of the Cox PH model. Ratio R: top-decile ratio, Equation (6). Max ECE: highest monthly expected calibration error, Equation (3). GGR covered: share of the GGR at risk covered by the 1,000 players with the highest value at risk, Equation (5); recency in round brackets. Red: condition failed. Source: `docs/backtest_v0_brand*.md`.

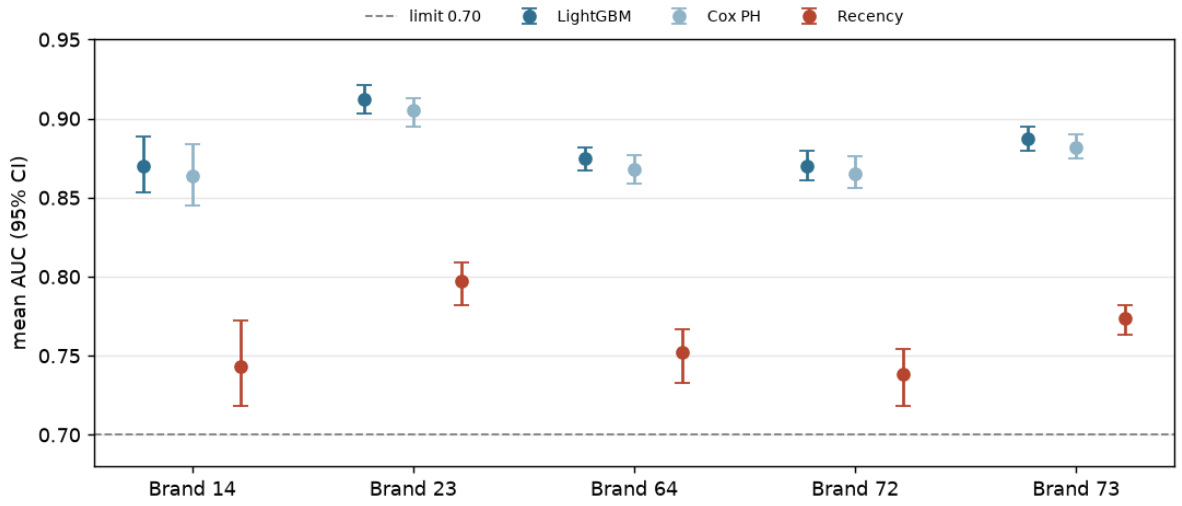


Figure 7: Mean backtest AUC by brand with 95% bootstrap intervals: LightGBM, Cox PH and recency alone. The dashed line is the pass limit of 0.70. Source: `docs/backtest_v0_brand*.md`.

The intervals of the model and of recency never overlap (Figure 7): the model beats recency at every cutoff, and its AUC is never below 0.82. The Cox PH model ranks almost as well as LightGBM. With the same 1,000 contacts, recency would cover 6% to 13% of the GGR at risk, and a random choice 3% to 9%. The only conditions that fail are ECE conditions.

4.6 What the model does better than recency

Recency is a fair comparison for the main job of the model: finding the players who are going to leave. Here the model is clearly better (Figure 8):

- **It finds more churners with the same contacts.** In brand 64, contacting the riskiest 10% of players finds 24% of all churners with the model and 17% with recency; at 20%, 46% against 35%; at 30%, 66% against 50%.
- **Its list is more accurate in every brand.** Of the riskiest 10% of players, 86% to 93% really churn with the model, against 68% to 81% with recency: 13 to 19 points more.
- **It gives what recency cannot give**: a calibrated probability, the 7 to 60-day horizons, risk bands and the main drivers of each player.

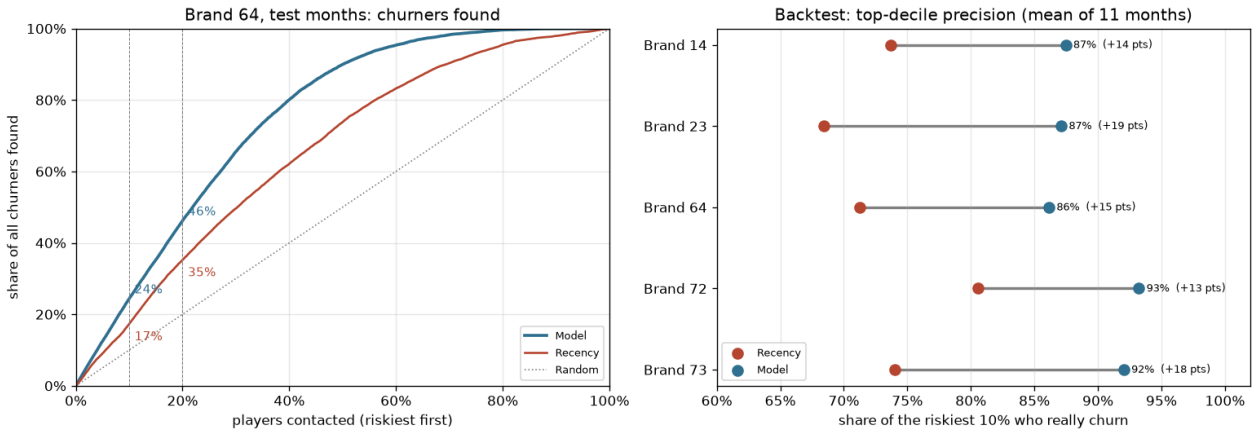


Figure 8: Model against recency. Left: share of all churners found when the players are contacted from the riskiest down (brand 64, test months July and August 2026; ties in recency broken at random). Right: mean top-decile precision over the 11 backtest months, for each brand. Source: MLflow run of `v63` and `docs/backtest_v0_brand*.md`.

4.7 Business value

The campaign replay measures money. At each cutoff, it takes the 1,000 players with the highest score and adds up the *GGR at risk reached*: the expected 30-day GGR (Equation (5)) of the contacted players who really churned. This is the revenue that a retention campaign could still act on.

The model ranks players by value at risk, $G_i\, p_i^{(60)}$, so it uses money, while recency does not. To see how much of the gain comes from the model and how much from simply using money, I also test a **value × recency rule**: a rule as simple as recency, but that also uses money,

$$s_{i,c}^{\text{val}} = G_i\, s_{i,c}^{\text{rec}}, \tag{7}$$

with $s_{i,c}^{\text{rec}}$ from Equation (4). It uses the same players, labels and cutoffs as the backtest.

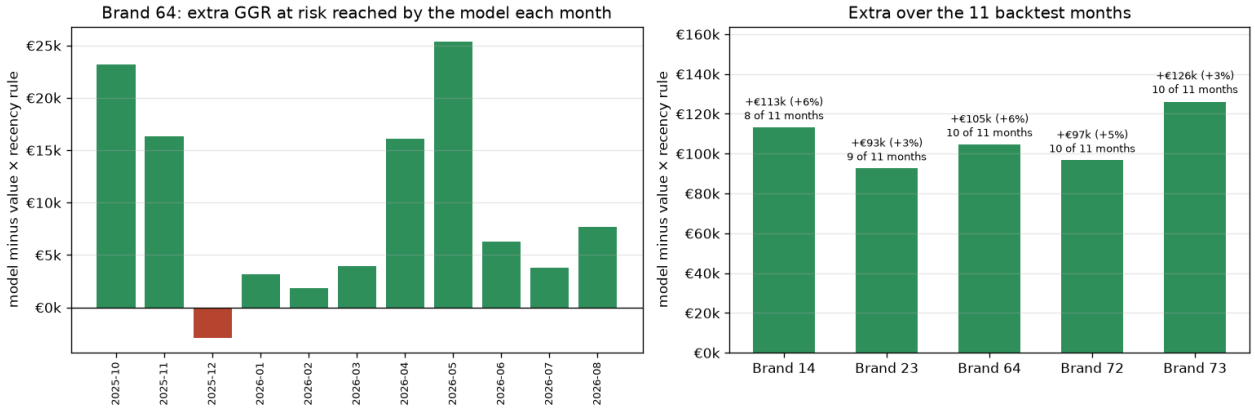


Figure 9: Extra GGR at risk reached by the model over the value × recency rule, with the same 1,000 contacts per month. Left: each backtest month for brand 64 (green: the model reaches more). Right: total over the 11 months for each brand, with the share of months where the model is better. Source: backtest replay files and backtest datasets.

Table 8: GGR at risk reached in the 11 backtest months. Each brand is independent.

Brand	Model	Value × recency	Recency	Model over value × recency
14	€1.97M	€1.86M	€0.37M	+€0.11M (+6%)
23	€3.47M	€3.38M	€0.47M	+€0.09M (+3%)
64	€1.86M	€1.75M	€0.28M	+€0.10M (+6%)
72	€1.94M	€1.84M	€0.28M	+€0.10M (+5%)
73	€4.62M	€4.49M	€0.44M	+€0.13M (+3%)

Note. GGR at risk reached: expected 30-day GGR (Equation (5), at the pace of the last 90 days) of the contacted players who churned, summed over the 11 monthly campaigns. Value × recency: Equation (7). Source: backtest replay files and backtest datasets.

- **Against recency alone**, the model reaches 5 to 10 times more GGR at risk (Table 8). Two things explain it: the model ranks churn better (AUC 0.11 to 0.13 higher, Table 7), and it uses money, which recency ignores. The second one is by far the bigger: the value × recency rule already reaches 94% to 97% of what the model reaches.
- **The model adds a small but steady extra** (Figure 9): 3% to 6% more GGR at risk than the value × recency rule, between €93k and €126k per brand over 11 months. It is better in 47 of the 55 months, and in at least 8 of 11 months in every brand.
- Both value-based rankings contact few of all the churners (2% to 5%), but the valuable ones. Recency finds players who are already gone; the value-based rankings find the valuable players who are about to leave.
- The GGR at risk reached is **not money saved**. The money saved is the GGR reached times the share of contacted churners that the campaign keeps. This share is not known yet: it needs a campaign with a control group.

What this means. For choosing who to contact, using value matters much more than the model. The model's own extra value is small in money: if a campaign kept 10% of the GGR it reaches, it would be about €10k per brand over 11 months against the value × recency rule. The model is still useful for other things that a rule cannot give: calibrated probabilities, the 7 to 60-day horizons, the risk bands and the drivers of each player.

4.8 Seasonal calibration

The calibration month is 2 to 3 months older than the scoring date, because the label needs 60 days. If the churn rate changes in between, the ranking of the players is still right, but the level of the probability is not. And the churn rate changes a lot from month to month (Figure 10, top): from 46% to 69% in brand 14, from 42% to 74% in brand 72, from 29% to 51% in brand 64 and from 19% to 42% in brand 23. Brand 73 is the most stable (39% to 49%) and has the best calibration.

Solution (`src/models/calibration_level.py`): on the scoring date, all the probabilities p_i of brand b are shifted in log-odds:

$$p_i' = \sigma\big(\mathrm{logit}(p_i) + \delta_b\big), \qquad \delta_b = \mathrm{median}_{m \in M}\ \delta_{b,m}, \qquad |\delta_b| \le 2, \tag{8}$$

where σ is the logistic function, M is the set of the last 3 closed months and the recent months with at least 14 days of data, and $\delta_{b,m}$ is the shift that makes the mean shifted probability of month m equal to its churn rate. The shift is the same for every player, so it does not change the ranking: the AUC and the top decile stay the same.

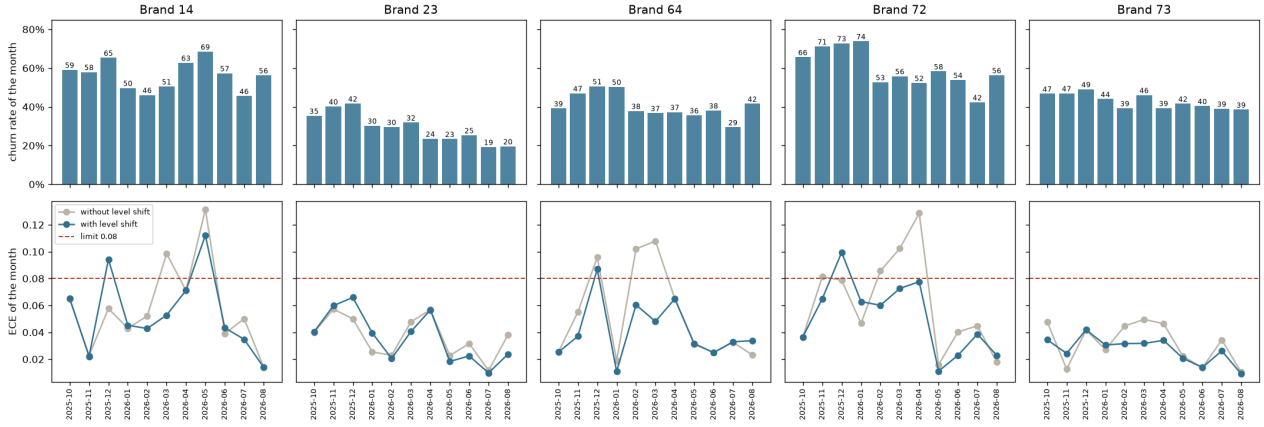


Figure 10: Backtest month by month. Top: churn rate of each test month. Bottom: ECE without the level shift (grey) and with it (blue); red line, limit of 0.08.

The ECE peaks match the jumps in churn. With the level shift, the months above 0.08 go down from 9 to 4 in the five brands: December 2025 in brands 64 and 72, and December and May in brand 14. In brand 72 alone they go down from 4 to 1. In these months the change happens after the scoring date, so no earlier data can show it. They will improve with one more year of history, which will allow a year-on-year seasonal term.

4.9 Top-decile condition

The plan asked for a top-decile precision of at least 1.5 times the recency rule. This is not possible: recency is already 68% to 81% precise on average, so even a perfect model cannot go above 1.24x to 1.46x (Figure 11). **I set the limit at 1.10x**, the strictest round value that the model passes with 95% confidence in the five brands (lower ends of the interval between 1.12 and 1.21). Details in `docs/decision_top_decile_condition.md`.

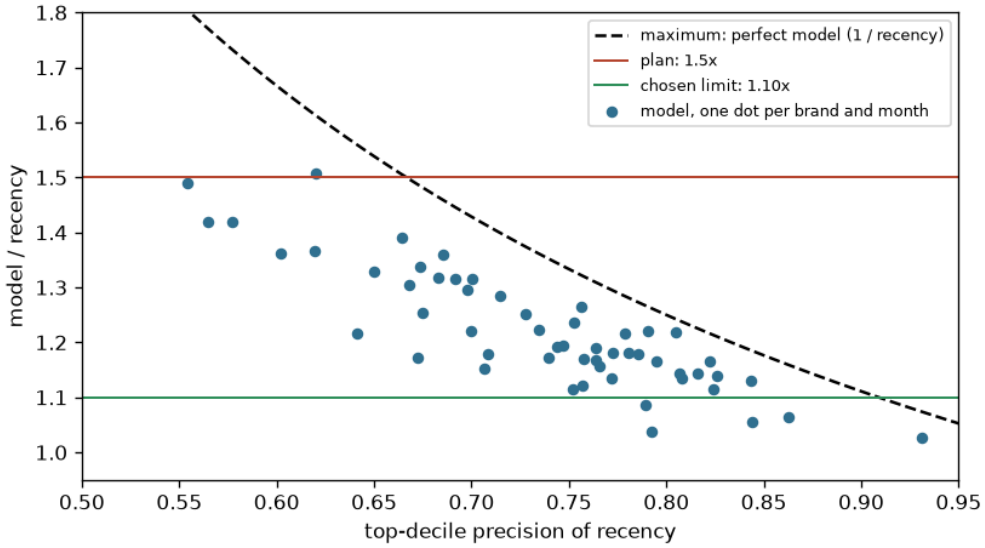


Figure 11: Each dot is one brand and one month. The dashed line is the highest possible value; 1.5x is almost always above it.

5 Validation of the signals table

I checked the 28 columns of `gld_player_signals_daily` that the features read, in five brands (`eda/signal_validation.py`, report in `docs/signal_validation.md`):

- **Recomputation**: 21 columns are rebuilt from the daily tables. A column is right if it matches (within 2% or 0.5) for 95% of the players.
- **Leak**: the same recomputation 7 days later. Two leaks made on purpose show that the test catches them.
- **Plausibility**: valid ranges, documented formulas and columns with values.

19 of the 28 columns pass every check in the five brands (Figure 12). The problems are in Table 9.



Figure 12: Failed checks by brand and monthly snapshot (green passes, red fails).

Table 9: Problems in the signals table and their effect.

Column	Problem	Effect
`days_since_bet`	0 for almost everyone until July 2026; from August, the recency of 7 days later	Not used; computed again
`tenure_days`	Wrong in the history; right from September 2026	Not used; computed again
`active_days_*`	Sometimes counts more days than the days with a bet, never fewer	Definition to confirm; two are model features
`tp_night_index`	Empty until August 2026; goes up to 4, so it is not a share	No model uses it
`deposit_frequency_score`	Empty before 2026	No model uses it

6 Decisions against the plan

Table 10: Differences with the delivery plan.

Topic	Plan	Done	Reason
Churn	30 days	60 days	With 30, 17% to 24% of churned players come back
Main score	`p_churn_30d`	`p_churn_60d`	Follows from the 60 days
Calibrator	ECE on test	Cross-validated ECE	The test is not used to decide
Calibration level	None	Estimated again when scoring	The calibration month is 2 to 3 months old
Top decile	$\geq$ 1.5x	$\geq$ 1.10x	1.5x is not possible
Feature store	S3 and Redis	Local	Read-only access to AWS

7 Conclusions and limitations

Conclusions

- **The model ranks churn risk well**: AUC of 0.87 to 0.91 over one year of backtest, between 0.11 and 0.13 above recency, and never below 0.82 in any month.
- **The money comes mostly from using value**: ranking by value reaches 5 to 10 times more GGR at risk than recency, and the model adds 3% to 6% over a simple value $\times$ recency rule. How much of it is kept depends on the campaign, which still has to be measured with a control group.
- **The ranking is stable; the level depends on the season**: the only failures are ECE failures, and the level shift reduces them from 9 to 4 months.
- **The reference configuration is enough**: it gives the same AUC as the optimised one and it is simpler.
- **The signals table is not reliable for recency and tenure**: the pipeline does not depend on it for these features.

Limitations

- 18 months of history, with no full previous year, and deposits only from March 2026.
- The backtest cutoffs are not independent, so the intervals are a bit optimistic.
- The top-decile KPI shows that the model beats recency, but not a simple rule with recency and frequency, which comes close.
- The campaign replay measures who is chosen, not the effect of the campaign.

- The feature store, the versioned snapshot and MLflow are local, because I only have read access to AWS.

A How to reproduce the project

The environment is pinned with `uv` (`pyproject.toml`, `uv.lock`, Python 3.13). `make test` passes 24 project tests and 23 tests of `eda/dq_lib`. MLflow is local (SQLite).

Table 11: Main commands.

Command	What it does
`make pipeline AS_OF=... [BRAND=...] [BACKTEST=1]`	The full pipeline on one date
`make backtest AS_OF=... BRAND=...`	11-month backtest
`make score AS_OF=... [MODEL_VERSION=n]`	Scores of the day
`make model-card BRAND=...`	HTML model card
`make validate-signals`	Validation of the signals table
`make test` / `make mlflow-ui`	Tests / MLflow interface

B Abbreviations and terms

AUC
Area under the ROC curve: probability that the model gives more risk to a player who leaves than to one who stays; 0.5 is random.

CI
Confidence interval (here, 95% bootstrap interval).

CRM
Customer relationship management team, who would contact the players.

Calibration
Turning the model score into a real probability.

Censoring
The churn cannot be seen yet because the data ends before.

C-index
The AUC of survival models.

Cox PH
Survival model: it estimates when churn comes.

ECE
Expected calibration error: mean difference between the predicted probability and the real rate, Equation (3).

GGR
Gross gaming revenue: money wagered minus money paid in wins.

IBS
Integrated Brier score: mean error of the survival curve; lower is better.

IQR, MAD
Interquartile range and median absolute deviation, two robust measures of spread.

KPI
Key performance indicator.

Log-odds
Logarithm of $p/(1-p)$; shifting it moves all probabilities without changing their order.

Point in time
Using only the data available on the cutoff date.

PSI
Population stability index: how much the distribution of a feature has changed.

ROC
Receiver operating characteristic curve: true positive rate against false positive rate.

Risk band
Decile of the churn probability within the brand and day.

SHAP
Shapley additive explanations: they split each player's score between the features.

Top decile
The 10% of players with the highest risk.

Value at risk
Expected GGR in 30 days times the churn probability, Equation (5).

Winsorisation
Capping extreme values at a percentile instead of deleting them.

References

Cox, D.R. (1972) 'Regression models and life-tables', *Journal of the Royal Statistical Society: Series B*, 34(2), pp. 187-220. doi:10.1111/j.2517-6161.1972.tb00899.x.

Davidson-Pilon, C. (2019) 'lifelines: survival analysis in Python', *Journal of Open Source Software*, 4(40), p. 1317. doi:10.21105/joss.01317.

Harrell, F.E., Califf, R.M., Pryor, D.B., Lee, K.L. and Rosati, R.A. (1982) 'Evaluating the yield of medical tests', *JAMA*, 247(18), pp. 2543-2546. doi:10.1001/jama.1982.03320430047030.

Kaplan, E.L. and Meier, P. (1958) 'Nonparametric estimation from incomplete observations', *Journal of the American Statistical Association*, 53(282), pp. 457-481. doi:10.1080/01621459.1958.10501452.